Arithmetic.h & Arithmetic.c

Deep Teaching Guide — What the file does · Every function explained · Syntax · Variables

Contents:

1 — What This File Does and Why It Exists

Arithmetic.h / Arithmetic.c provides ELEMENT-WISE arithmetic operations on tensors. Element-wise means: apply an operation to each pair of corresponding elements, one element at a time.

For example, adding two tensors A and B means: result[0]=A[0]+B[0], result[1]=A[1]+B[1], ..., result[n]=A[n]+B[n]. Each position is computed independently.

The big design idea — function pointers as the operation:

Instead of writing a separate addTensors(), subtractTensors(), multiplyTensors() function for every possible operation, this file writes ONE general function and lets you pass the operation as a parameter. The operation is passed as a FUNCTION POINTER — a variable that holds the address of a function like add, subtract, or multiply.

```
// You define the operation as a small function:
int32_t myAdd(int32_t a, int32_t b) { return a + b; }
int32_t mySub(int32_t a, int32_t b) { return a - b; }

// Then pass it to the general function:
int32PointWiseArithmetic(tensorA, tensorB, myAdd, outputTensor); // adds
int32PointWiseArithmetic(tensorA, tensorB, mySub, outputTensor); // subtracts
```

The four operation patterns:

Pattern	What it does	Example
`PointWise`	Apply op to each pair of elements from two tensors	C[i] = op(A[i], B[i])
`PointWiseInplace`	Same, but result written back into aTensor	A[i] = op(A[i], B[i])
`ElementWithTensor`	Apply op to each element of tensor with a single scalar x	C[i] = op(A[i], x)
`ElementWithTensorInplace`	Same, written back into tensor	A[i] = op(A[i], x)

Each pattern exists in two data-type versions:

Version	For tensors of type	Function pointer type
`int32...`	INT32 (32-bit integers)	int32ElementArithmeticFunc_t
`float...`	FLOAT32 (32-bit floats)	floatElementArithmeticFunc_t

2 — Key Concepts Before Reading the Code

Element-wise (point-wise) operation — An operation applied independently to each pair of corresponding positions in two tensors. Position 0 of A is combined with position 0 of B. Position 1 of A with position 1 of B. Etc. Both tensors must have the same shape.
Think of it as: Adding two lists: [1,2,3] + [10,20,30] = [11,22,33]

Function pointer parameter — A function can receive another function as an argument, via a function pointer. This lets you write ONE general loop and plug in ANY operation. The function pointer type int32ElementArithmeticFunc_t means: 'a pointer to any function that takes two int32_t and returns one int32_t'.
Think of it as: A calculator that accepts 'which button to press' as input — the same hardware, different operation.

In-place vs out-of-place — Out-of-place: result is written into a SEPARATE outputTensor. aTensor and bTensor are unchanged. In-place: result is written BACK into aTensor itself. aTensor is modified. In-place saves memory (no extra tensor needed) but destroys the original values.
Think of it as: Out-of-place = drafting on a new sheet. In-place = editing the original document.

Flat index (raw index) — A tensor's data is stored as a flat array of bytes in memory. A flat index is a single number that addresses one position in that flat array. For a 2D tensor of shape [3,4]: flat index 0 = row 0 col 0, flat index 5 = row 1 col 1.
Think of it as: Page number in a book — one number reaches any page.

Multi-dimensional indices — A set of numbers, one per dimension, that addresses a specific element. For shape [3,4]: indices [1,2] means row 1, column 2. The index calculation functions convert between flat and multi-dim.
Think of it as: Row and column coordinates on a grid.

orderOfDimensions (transpose support) — A tensor can be transposed without moving its data — instead, the shape's orderOfDimensions array is changed. orderOfDimensions = [0,1] means normal order. orderOfDimensions = [1,0] means transposed (rows and columns swapped). calcElementIndexByIndices uses this to find the correct byte position.
Think of it as: Rotating a grid by changing which axis you read along — the numbers don't move.

sizeof(type) — sizeof is a C operator that returns the number of BYTES a type occupies. sizeof(int32_t) = 4. sizeof(float) = 4. sizeof(double) = 8. Used here to compute byte offsets: to reach element i, skip i * sizeof(type) bytes.

& operator (address-of) — The & placed BEFORE a variable gives its memory address. &tensor-;>data[byteIndex] means: give me the address of the byte at position byteIndex in the data array. This address is then passed to readBytesAsInt32() or readBytesAsFloat() so those functions know WHERE to read from.
Think of it as: & is like saying 'the location of' — not the value, but where it lives in memory.

! operator (logical NOT) — ! reverses a boolean: !true = false, !false = true. !doDimensionsMatch(a,b) means 'if the dimensions do NOT match'. Used as a guard: if they don't match, print error and stop.

bool return type — bool is a true/false type from <stdbool.h>. A function returning bool returns either true (1) or false (0). doDimensionsMatch returns true if shapes are equal, false if not.

3 — Arithmetic.h — Header Line by Line

```
#ifndef ENV5_RUNTIME_TENSOR_MATH_H
#define ENV5_RUNTIME_TENSOR_MATH_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include "Tensor.h"

typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);
typedef float (*floatElementArithmeticFunc_t)(float a, float b);

bool doDimensionsMatch(tensor_t *a, tensor_t *b);
void orderDims(tensor_t *tensor, size_t *orderedDims);
size_t getDimensionsByIndex(tensor_t *tensor, size_t index);
size_t calcTensorIndexByIndices(size_t numberOfDimensions, size_t *dimensions, size_t *indices);
void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t *indices);
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims,
size_t *indices, size_t *orderOfDimensions);

void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor);
void floatPointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor);
void int32PointWiseArithmeticInplace(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc);
void floatPointWiseArithmeticInplace(tensor_t *aTensor, tensor_t *bTensor,
floatElementArithmeticFunc_t arithmeticFunc);
void int32ElementWithTensorArithmetic(tensor_t *aTensor, int32_t x,
int32ElementArithmeticFunc_t arithmeticFunc,
tensor_t *outputTensor);
void floatElementWithTensorArithmetic(tensor_t *aTensor, float x,
floatElementArithmeticFunc_t arithmeticFunc,
tensor_t *outputTensor);
void int32ElementWithTensorArithmeticInplace(tensor_t *aTensor, int32_t x,
int32ElementArithmeticFunc_t arithmeticFunc);
void floatElementWithTensorArithmeticInplace(tensor_t *aTensor, float x,
floatElementArithmeticFunc_t arithmeticFunc);
```

```
39  #endif
```

Line 1–2 (header guard)

#ifndef ENV5_RUNTIME_TENSOR_MATH_H — 'if not defined'. This prevents the header from being processed twice if multiple .c files include it. **#define** marks it as seen. **#endif** at line 38 closes the block.

Line 4–6 (system includes)

<stdbool.h> — provides the **bool** type and the values **true** / **false**. Without this include, writing 'bool' would be an unknown type.

<stddef.h> — provides **size_t** (the unsigned type for sizes and array indices).

<stdint.h> — provides fixed-size integer types like **int32_t** (always exactly 32 bits), **uint8_t** (always 8 bits), etc. Guarantees consistent sizes across all platforms.

Line 10 (int32ElementArithmeticFunc_t — the most important line)

typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);

Break it apart:

typedef — create a new type name.

int32_t at the start — any function of this type RETURNS an int32_t.

(*int32ElementArithmeticFunc_t) — the new type name. The * inside means it is a POINTER to a function.

(int32_t a, int32_t b) — any function of this type takes two int32_t as input.

So: int32ElementArithmeticFunc_t is a type for 'any function that takes two int32_t and gives back one int32_t'. You could use: add, subtract, multiply, max, min — anything with that signature.

After this typedef, you can declare a variable of this type: **int32ElementArithmeticFunc_t fn = myAdd;** and call it as **fn(3, 4)**.

Line 11 (floatElementArithmeticFunc_t)

Same pattern as line 10, but for **float** instead of **int32_t**. Used by the float versions of the arithmetic functions.

4 — getDimensionsByIndex

Returns the SIZE of a dimension at a given logical index. It respects the orderOfDimensions — so if the tensor is transposed, index 0 still refers to the logical first dimension, not the stored first dimension.

```
1  size_t getDimensionsByIndex(tensor_t *tensor, size_t index) {
2  size_t numberOfDims = tensor->shape->numberOfDimensions;
3  for (size_t i = 0; i < numberOfDims; i++) {
4  if (tensor->shape->orderOfDimensions[i] == index) {
5  return tensor->shape->dimensions[i];
6  }
7  }
8  PRINT_ERROR("Tensor doesn't have %lu dimensions!", index);
9  exit(1);
10 }
```

Line 1 (return type size_t, parameter size_t index)

size_t is the return type — the function returns a dimension size (always non-negative).

tensor_t *tensor — pointer to the tensor to query.

size_t index — which logical dimension to look up. 0 = first, 1 = second, etc.

Line 2 (reading numberOfDimensions)

tensor->shape->numberOfDimensions — follow two pointers: tensor→shape→numberOfDimensions. Gives the total number of dimensions (e.g. 2 for a matrix).

Line 3–6 (the search loop)

tensor->shape->orderOfDimensions[i] — the orderOfDimensions array stores which logical dimension each physical position corresponds to. For a normal tensor: orderOfDimensions = [0, 1]. For a transposed tensor: orderOfDimensions = [1, 0].

The loop checks each position: 'does this physical position correspond to the requested logical index?' When it finds a match, it returns **tensor->shape->dimensions[i]** — the SIZE of that dimension.

Example: tensor shape [3,4], transposed so orderOfDimensions=[1,0]. Asking for index=0 (first logical dim): loop checks i=0: orderOfDimensions[0]=1 ≠ 0. Loop checks i=1: orderOfDimensions[1]=0 == 0. Returns dimensions[1]=4. So the first logical dimension has size 4 (the column count, since rows/cols are swapped).

Line 8–9 (error if not found)

If the loop finishes without finding the index, the requested dimension does not exist. **PRINT_ERROR** prints a message showing which index was invalid. **exit(1)** stops the program immediately. The **%lu** format specifier is for **unsigned long** (how size_t is printed on most systems).

5 — orderDims

Fills an array with the dimension sizes in LOGICAL order (0, 1, 2, ...), regardless of how the tensor is stored internally or whether it is transposed.

```
1  void orderDims(tensor_t *tensor, size_t *orderedDims) {
2  for (size_t i = 0; i < tensor->shape->numberOfDimensions; i++) {
3  orderedDims[i] = getDimensionsByIndex(tensor, i);
4  }
5  }
```

Line 1 (size_t *orderedDims — output parameter)

size_t *orderedDims — a pointer to an array that the CALLER provides. The function writes into this array. This is how C functions return multiple values — you pass a pointer to where you want the results written.

The caller declares an array first: **size_t myDims[n];** then passes it: **orderDims(tensor, myDims);** After the call, myDims[0] = first dimension size, myDims[1] = second, etc.

Line 2–4 (the loop)

Loops i from 0 to numberOfDimensions-1. For each i, calls getDimensionsByIndex(tensor, i) to get the size of logical dimension i, and stores it at orderedDims[i].

Example: tensor shape [3,4], transposed (orderOfDimensions=[1,0]). i=0: getDimensionsByIndex returns 4 (the logical first dim after transpose). i=1: returns 3. orderedDims = [4, 3].

6 — doDimensionsMatch

Checks whether two tensors have exactly the same shape. Returns true if they match, false if not. Also prints an error and stops the program if the number of dimensions differs.

```
1  bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
2  size_t aNumberOfDims = a->shape->numberOfDimensions;
3  size_t bNumberOfDims = b->shape->numberOfDimensions;
4  if (aNumberOfDims != bNumberOfDims) {
5  PRINT_ERROR("Rank mismatch: %lu vs %lu\n", aNumberOfDims, bNumberOfDims);
6  exit(1);
7  }
8  size_t aOrderedDims[aNumberOfDims];
9  size_t bOrderedDims[bNumberOfDims];
10 orderDims(a, aOrderedDims);
11 orderDims(b, bOrderedDims);
12 for (size_t i = 0; i < aNumberOfDims; i++) {
13 if (aOrderedDims[i] != bOrderedDims[i]) {
14 return false;
15 }
```

```
16  }
17  return true;
18  }
```

Line 1 (bool return type)

bool — this function returns either **true** or **false**. The caller uses the result to decide what to do: **if (!doDimensionsMatch(a,b))** — if they DON'T match, print error and stop.

Line 2–7 (checking number of dimensions first)

aNumberOfDims — how many dimensions tensor a has. E.g. 2 for a matrix, 1 for a vector.

if (aNumberOfDims != bNumberOfDims) — if a has 2 dimensions and b has 3, they cannot possibly match. This is called a RANK MISMATCH. The function prints an error and calls exit(1) — stops the program. This is stricter than returning false — a rank mismatch is a programming error, not a data mismatch.

Line 8–11 (VLA arrays and orderDims)

size_t aOrderedDims[aNumberOfDims] — a VLA (Variable Length Array) on the stack. Its size is aNumberOfDims, which is only known at runtime.

orderDims(a, aOrderedDims) — fills the array with dimension sizes in logical order. This is important because the two tensors might be stored with different internal orderings (one might be transposed). Using orderDims gives both their logical sizes for a fair comparison.

Line 12–16 (comparing each dimension)

Loop over each dimension index. If any dimension size differs (e.g. a is [3,4] and b is [3,5]), return **false** immediately. If all match, the loop finishes and we reach **return true** on line 17.

7 — calcTensorIndexByIndices — Multi-dim Indices → Flat Index

Given multi-dimensional indices (e.g. row=1, col=2 for a 2D tensor), this function computes the flat position in the data array. This is the standard row-major formula.

```
1  size_t calcTensorIndexByIndices(size_t numberOfDimensions, size_t *dimensions, size_t *indices) {
2  size_t index = indices[numberOfDimensions - 1];
3  size_t offset = dimensions[numberOfDimensions - 1];
4  for (int i = (int)numberOfDimensions - 2; i >= 0; i--) {
5  index += indices[i] * offset;
6  offset *= dimensions[i];
7  }
8  return index;
9  }
```

Line 1 (three parameters — all arrays passed as pointers)

numberOfDimensions — how many dimensions the tensor has.

size_t *dimensions — pointer to the array of dimension sizes. E.g. for shape [3,4]: dimensions[0]=3, dimensions[1]=4.

size_t *indices — pointer to the array of index values. E.g. indices[0]=1, indices[1]=2 means 'row 1, column 2'.

In C, arrays are passed as pointers — **size_t *dimensions** and **size_t dimensions[]** mean the same thing in a function parameter.

Line 2–3 (start from the last dimension)

The formula starts from the innermost (last) dimension.

index = indices[numberOfDimensions-1] — start with the last index. For shape [3,4] with indices [1,2]: index = indices[1] = 2.

offset = dimensions[numberOfDimensions-1] — the stride of the current dimension. offset = dimensions[1] = 4.

Line 4–7 (the accumulation loop — row-major formula)

The loop goes from dimension numberOfDimensions-2 DOWN to 0 (backward). The **int i** (not size_t) allows i >= 0 as a condition — size_t is unsigned so it cannot go below 0, which would cause an infinite loop.

index += indices[i] * offset — add this dimension's contribution. How far along is element [i]? It is indices[i] × (product of all later dimensions).

offset *= dimensions[i] — grow the offset for the next (outer) dimension.

Example: shape [3,4], indices [1,2].

Start: index=2, offset=4.

i=0: index += indices[0] * offset = 1*4 = 4 → index=6. offset *= 3 = 12.

Result: 6. Check: row 1 × 4 columns + col 2 = 4+2 = 6. ✓

→ **Row-major formula:** flatIndex = row × numCols + col. For [3,4] tensor: element [1][2] = 1×4+2 = 6. Generalises to any number of dimensions.

8 — calcIndicesByRawIndex — Flat Index → Multi-dim Indices

The INVERSE of calcTensorIndexByIndices. Given a flat position number (0, 1, 2, ...), compute the multi-dimensional indices (row, column, ...). Used in the point-wise loop to determine which logical element corresponds to each flat position.

```
1  void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t
   *indices) {
2  size_t offset = 1;
3  for (size_t i = 0; i < numberOfDims; i++) {
4  offset *= dims[i];
5  }
6  size_t restIndex = rawIndex;
7  for (size_t i = 0; i < numberOfDims; i++) {
8  offset /= dims[i];
9  indices[i] = restIndex / offset;
10 restIndex -= indices[i] * offset;
11 }
12 }
```

Line 1 (parameters)

rawIndex — the flat position to decode. E.g. 6 in a [3,4] tensor.

size_t *indices — OUTPUT array. The caller provides it and the function fills it. After the call: indices[0]=row, indices[1]=col, etc.

Line 2–5 (compute total elements = initial offset)

offset starts at 1 and is multiplied by each dimension. After this loop, offset = numberOfElements = dims[0] × dims[1] × ... × dims[n-1]. For shape [3,4]: offset = 3×4 = 12.

Line 6–11 (decode each dimension with division)

restIndex starts as rawIndex. We peel off one dimension at a time.

offset /= dims[i] — divide by current dimension to get the stride of the remaining dimensions. For [3,4] at i=0: offset = 12/3 = 4. At i=1: offset = 4/4 = 1.

indices[i] = restIndex / offset — how many complete 'blocks' of size offset fit? That is the index for this dimension. For rawIndex=6, i=0: 6/4 = 1 (integer division). indices[0]=1.

restIndex -= indices[i] * offset — subtract the contribution of this dimension. restIndex = 6 - 1×4 = 2.

At i=1: offset=1. indices[1] = 2/1 = 2. restIndex = 2 - 2×1 = 0.

Result: indices = [1, 2]. Meaning: row 1, column 2. ✓

9 — calcElementIndexByIndices — Transpose-Aware Element Position

Converts multi-dimensional indices to a flat element position, taking into account the tensor's orderOfDimensions (transpose). This is the function that makes arithmetic work correctly on transposed tensors.

```
1  size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims,
2  size_t *indices, size_t *orderOfDimensions) {
3  size_t offset = 1;
4  for (size_t i = 0; i < numberOfDims; i++) {
5  offset *= dims[i];
6  }
7  size_t orderedIndices[numberOfDims];
8  for (size_t d = 0; d < numberOfDims; d++) {
9  orderedIndices[orderOfDimensions[d]] = indices[d];
10 }
11 size_t outputIndex = 0;
12 for (size_t i = 0; i < numberOfDims; i++) {
13 offset /= dims[i];
14 outputIndex += orderedIndices[i] * offset;
15 }
16 return outputIndex;
17 }
```

Line 1–2 (parameters)

size_t *dims — the physical dimensions array (as stored, before transpose).

size_t *indices — the logical indices (row, col in the logical view).

size_t *orderOfDimensions — the transpose permutation. Normal: [0,1]. Transposed: [1,0].

Line 3–6 (compute total offset — same as before)

Same as calcIndicesByRawIndex: multiply all dimensions to get total elements.

Line 7–10 (re-ordering the indices using orderOfDimensions)

This is the key step for transpose support.

orderedIndices[orderOfDimensions[d]] = indices[d] — this re-maps the logical indices to physical positions.

Example: tensor shape [3,4], transposed so orderOfDimensions=[1,0]. Logical indices [1,2] (row 1, col 2 in the TRANSPOSED view):

d=0: orderedIndices[orderOfDimensions[0]] = orderedIndices[1] = indices[0] = 1.

d=1: orderedIndices[orderOfDimensions[1]] = orderedIndices[0] = indices[1] = 2.

orderedIndices = [2, 1] (swapped).

Now orderedIndices holds the physical coordinates in the stored layout.

Line 11–15 (compute flat position from physical indices)

Same row-major formula as calcTensorIndexByIndices, but using orderedIndices (physical). For the example: outputIndex = 2×4 + 1×1 = 9.

This is the physical byte position in the data array — the element that logically sits at row 1, col 2 of the transposed view.

10 — int32PointWiseArithmetic — Apply Operation to Two Tensors

The core function. Loops over every element position, reads the corresponding element from each tensor, applies the operation function, and writes the result to the output tensor. Works correctly even for transposed tensors.

```
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc,
tensor_t *outputTensor) {
if (!doDimensionsMatch(aTensor, bTensor)) {
PRINT_ERROR("Dimensions don't match!");
exit(1);
}
size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
size_t bytesPerElement = sizeof(int32_t);
size_t numberOfDims = aTensor->shape->numberOfDimensions;
size_t *aDims = aTensor->shape->dimensions;
size_t *bDims = bTensor->shape->dimensions;
size_t aOrderedDims[numberOfDims];
size_t bOrderedDims[numberOfDims];
orderDims(aTensor, aOrderedDims);
orderDims(bTensor, bOrderedDims);
for (size_t i = 0; i < numberOfElements; i++) {
size_t aIndices[numberOfDims];
calcIndicesByRawIndex(numberOfDims, aDims, i, aIndices);
size_t aElementIndex = calcElementIndexByIndices(numberOfDims, aDims,
aIndices, aTensor->shape->orderOfDimensions);
size_t bIndices[numberOfDims];
calcIndicesByRawIndex(numberOfDims, bDims, i, bIndices);
size_t bElementIndex = calcElementIndexByIndices(numberOfDims, bDims,
bIndices, bTensor->shape->orderOfDimensions);
size_t aByteIndex = aElementIndex * bytesPerElement;
size_t bByteIndex = bElementIndex * bytesPerElement;
int32_t aValue = readBytesAsInt32(&aTensor->data[aByteIndex]);
int32_t bValue = readBytesAsInt32(&bTensor->data[bByteIndex]);
int32_t result = arithmeticFunc(aValue, bValue);
size_t outputByteIndex = i * bytesPerElement;
writeInt32ToByteArray(result, &outputTensor->data[outputByteIndex]);
}
}
```

Line 1–3 (parameters)

tensor_t *aTensor — first input tensor.

tensor_t *bTensor — second input tensor. Must have the same shape as aTensor.

int32ElementArithmeticFunc_t arithmeticFunc — the OPERATION to apply. This is a function pointer. The caller passes the address of a function like add or multiply. Inside the loop, this function is called for each element pair.

tensor_t *outputTensor — where the results are written. Must be pre-allocated by the caller with enough space.

Line 4–7 (!doDimensionsMatch guard)

! is the logical NOT operator. !doDimensionsMatch(aTensor, bTensor) means 'if the dimensions do NOT match'. You cannot add a [3,4] tensor to a [2,5] tensor — there is no sensible meaning. The guard catches this mistake early with a clear error message.

Line 8–9 (element count and bytes per element)

numberOfElements — total count of values (e.g. 12 for a [3,4] tensor).

sizeof(int32_t) = 4. This is the number of bytes each integer occupies. To reach element i in the raw byte array, skip i × 4 bytes.

Line 10–16 (setup: dims and ordered dims)

aDims = pointer to aTensor's physical dimensions array. Not a copy — just a pointer alias. Cheap.

aOrderedDims — VLA filled by orderDims. Contains dimension sizes in logical order. Not actually used further in this function (only used for the doDimensionsMatch check above), but available for debugging.

Line 17 (the main loop: i iterates over flat positions)

for (size_t i = 0; i < numberOfElements; i++) — i goes 0, 1, 2, ... up to total-1. i is the logical flat position. For a [3,4] tensor: 0 to 11.

Line 18–25 (the index translation — the hardest part)

For each flat position i, we need to find the correct byte in each tensor. This is non-trivial because the tensors might be transposed differently.

Step 1: **calcIndicesByRawIndex(numberOfDims, aDims, i, aIndices)** — convert flat position i into multi-dim indices for tensor A. E.g. i=6 in [3,4] → aIndices=[1,2] (row 1, col 2).

Step 2: **calcElementIndexByIndices(numberOfDims, aDims, aIndices, aTensor->shape->orderOfDimensions)** — convert those logical indices into a physical element position, accounting for any transpose. Returns aElementIndex.

The same two steps are done for tensor B.

Why decode and re-encode? Because if A is transposed, its orderOfDimensions is [1,0], so the physical element at logical position [1,2] is different from a non-transposed tensor.

Line 26–27 (byte index from element index)

aByteIndex = aElementIndex * bytesPerElement — each element occupies 4 bytes. Multiply by 4 to get the byte offset. If aElementIndex = 6: aByteIndex = 24 (bytes 24–27 in the raw buffer).

Line 28–29 (reading values from raw bytes)

&aTensor-;>data[aByteIndex] — the & gives the ADDRESS of byte aByteIndex in the data array. This is passed to readBytesAsInt32(), which reads 4 bytes from that address and reconstructs an int32_t.

aValue and **bValue** are now proper int32_t values — ready to be used.

Line 30 (calling the operation through the function pointer)

arithmeticFunc(aValue, bValue) — this calls whatever function was passed. If the caller passed myAdd, this computes aValue + bValue. If the caller passed myMultiply, it computes aValue * bValue. The function pointer makes this completely generic.

int32_t result — stores the returned value.

Line 31–32 (writing result to output — note the simpler index)

outputByteIndex = i * bytesPerElement — the OUTPUT is always written in flat order, starting from byte 0. No transpose remapping needed — the output is a fresh non-transposed tensor. Element 0 → byte 0. Element 1 → byte 4. Etc.

writeInt32ToByteArray(result, &outputTensor-;>data[outputByteIndex]) — writes the 4 bytes of the result integer into the output buffer at the correct position.

→ **Usage example — adding two tensors:**

```
int32_t myAdd(int32_t a, int32_t b) { return a + b; }
int32PointWiseArithmetic(tensorA, tensorB, myAdd, outputTensor);
```

11 — int32PointWiseArithmeticInplace — In-Place Version

Identical to int32PointWiseArithmetic except the result is written back into aTensor. No separate outputTensor is needed. aTensor is modified in-place.

```
1   /* Important: result will be written into aTensor datafield */
2   void int32PointWiseArithmeticInplace(tensor_t *aTensor, tensor_t *bTensor,
3   int32ElementArithmeticFunc_t arithmeticFunc) {
4   /* ... same setup ... */
5   for (size_t i = 0; i < numberOfElements; i++) {
6   /* ... same index calculation ... */
7   int32_t aValue = readBytesAsInt32(&aTensor->data[aByteIndex]);
8   int32_t bValue = readBytesAsInt32(&bTensor->data[bByteIndex]);
9   int32_t result = arithmeticFunc(aValue, bValue);
10  size_t outputByteIndex = i * bytesPerElement;
11  writeInt32ToByteArray(result, &aTensor->data[outputByteIndex]); // <-- aTensor!
12  }
13  }
```

Line 1 (comment — tells you the key difference)

The comment '/* Important: result will be written into aTensor datafield */' warns you that aTensor will be MODIFIED. Its original values are overwritten.

This is the ONLY difference from int32PointWiseArithmetic: the last parameter (outputTensor) is removed, and on line 11, **&aTensor-;>data[outputByteIndex]** is used instead of &outputTensor-;>data[...].

Line 11 (writing to aTensor instead of outputTensor)

writeInt32ToByteArray(result, &aTensor-;>data[outputByteIndex]) — result is written into aTensor's own buffer at position i (in sequential order). Note: we read from aByteIndex (which accounts for transpose) but write to outputByteIndex (which is sequential from 0). This means the transposed layout is 'unwound' in the process — aTensor ends up in normal sequential order after the operation.

★ **When to use Inplace:** When you do not need the original aTensor values after the operation and want to save memory (no extra tensor needed). **When to use the normal version:** When you need to keep aTensor unchanged.

12 — int32ElementWithTensorArithmetic — Scalar Applied to Every Element

Instead of combining two tensors, this applies an operation between each element of a tensor and a single scalar value x. Example: multiply every element by 2, or add 5 to every element.

```
1  void int32ElementWithTensorArithmetic(tensor_t *tensor, int32_t x,
2  int32ElementArithmeticFunc_t arithmeticFunc,
3  tensor_t *outputTensor) {
4  size_t numberOfElements = calcNumberOfElementsByTensor(tensor);
5  size_t bytesPerElement = sizeof(int32_t);
6  for (size_t i = 0; i < numberOfElements; i++) {
7  size_t byteIndex = i * bytesPerElement;
8  int32_t currentElement = readBytesAsInt32(&tensor->data[byteIndex]);
9  int32_t result = arithmeticFunc(currentElement, x);
10 writeInt32ToByteArray(result, &outputTensor->data[byteIndex]);
11 }
12 }
```

Line 1–3 (parameters — note int32_t x)

tensor_t *tensor — the input tensor. Every element will be combined with x.

int32_t x — a single scalar value (one integer, not a tensor). This is NOT a pointer — it is the value itself passed directly. Example: x=2 to multiply everything by 2, or x=5 to add 5 to everything.

arithmeticFunc — the operation. arithmeticFunc(element, x) is called for each element.

outputTensor — where results are written.

Line 7 (byteIndex — simpler than PointWise)

byteIndex = i * bytesPerElement — this function iterates sequentially without transpose remapping. Why? Because there is only ONE tensor to read from (no second tensor with a different transpose), so a simple sequential scan is correct. Element 0 → bytes 0–3. Element 1 → bytes 4–7. Etc.

Line 8 (reading the current element)

&tensor-;>data[byteIndex] — & gives the address of byte byteIndex. **readBytesAsInt32** reads 4 bytes from there and returns an int32_t. **currentElement** now holds the integer value at position i.

Line 9 (calling the operation)

arithmeticFunc(currentElement, x) — calls the function passed by the caller. The first argument is the element from the tensor. The second is x (the scalar). So if arithmeticFunc is multiply: result = currentElement * x.

Usage example: multiply all elements by 3.

int32_t myMul(int32_t a, int32_t b) { return a * b; }

int32ElementWithTensorArithmetic(myTensor, 3, myMul, output);

int32ElementWithTensorArithmeticInplace

Same function, but the result is written back into tensor itself. Only difference: the last parameter (outputTensor) is removed, and the write goes to **&tensor-;>data[byteIndex]** instead of outputTensor.

```
1  void int32ElementWithTensorArithmeticInplace(tensor_t *tensor, int32_t x,
2  int32ElementArithmeticFunc_t arithmeticFunc) {
3  size_t numberOfElements = calcNumberOfElementsByTensor(tensor);
4  size_t bytesPerElement = sizeof(int32_t);
5  for (size_t i = 0; i < numberOfElements; i++) {
6  size_t byteIndex = i * bytesPerElement;
7  int32_t currentElement = readBytesAsInt32(&tensor->data[byteIndex]);
8  int32_t result = arithmeticFunc(currentElement, x);
9  writeInt32ToByteArray(result, &tensor->data[byteIndex]); // same buffer!
10 }
11 }
```

Line 9 (reading and writing the same position)

readBytesAsInt32(&tensor-;>data[byteIndex]) — reads element i.

writeInt32ToByteArray(result, &tensor-;>data[byteIndex]) — writes result back to the SAME byteIndex.

This is safe because we read before we write — the old value is in 'currentElement' before it is overwritten.

13 — Float Versions of All Functions

Every int32 function has an identical float counterpart. The logic is exactly the same — only the data type changes. Here are the differences:

int32 version	float version	What changes
`int32PointWiseArithmetic`	`floatPointWiseArithmetic`	int32_t → float, readBytesAsInt32 → readBytesAsFloat, writeInt32ToByteArray → writeFloatToByteArray, sizeof(int32_t) → sizeof(float)
`int32PointWiseArithmeticInplace`	`floatPointWiseArithmeticInplace`	Same type substitutions
`int32ElementWithTensorArithmetic`	`floatElementWithTensorArithmetic`	int32_t x → float x, same type substitutions
`int32ElementWithTensorArithmeticInplace`	`floatElementWithTensorArithmeticInplace`	Same
`int32ElementArithmeticFunc_t`	`floatElementArithmeticFunc_t`	Function pointer type — int32_t(int32_t,int32_t) → float(float,float)

Example — using the float version to multiply a FLOAT32 tensor by 0.5:

```
// Define your operation as a small function
float halfOf(float a, float b) { return a * b; }

// Call the float version with scalar 0.5f
floatElementWithTensorArithmeticInplace(myFloatTensor, 0.5f, halfOf);
// Now every element in myFloatTensor is halved
```

Example — adding two FLOAT32 tensors:

```
float myAdd(float a, float b) { return a + b; }

floatPointWiseArithmetic(tensorA, tensorB, myAdd, outputTensor);
// outputTensor[i] = tensorA[i] + tensorB[i] for all i
```

14 — Quick Reference Card

All functions at a glance:

Function	What it does	Output goes to	Data type
`getDimensionsByIndex(tensor, i)`	Size of logical dimension i (transpose-aware)	return value	any
`orderDims(tensor, arr)`	Fill arr with logical dimension sizes	arr[] (caller-provided)	any
`doDimensionsMatch(a, b)`	Check two tensors have same shape	bool return	any
`calcTensorIndexByIndices(n, dims, idx)`	Multi-dim indices → flat index	return value	any
`calcIndicesByRawIndex(n, dims, raw, idx)`	Flat index → multi-dim indices	idx[] (caller-provided)	any
`calcElementIndexByIndices(n, dims, idx, order)`	Indices → physical element pos (transpose-aware)	return value	any
`int32PointWiseArithmetic(a, b, fn, out)`	op(a[i], b[i]) for all i	outputTensor	INT32
`floatPointWiseArithmetic(a, b, fn, out)`	Same for floats	outputTensor	FLOAT32
`int32PointWiseArithmeticInplace(a, b, fn)`	op(a[i], b[i]) → written back into a	aTensor	INT32
`floatPointWiseArithmeticInplace(a, b, fn)`	Same for floats	aTensor	FLOAT32
`int32ElementWithTensorArithmetic(t, x, fn, out)`	op(t[i], x) for all i	outputTensor	INT32
`floatElementWithTensorArithmetic(t, x, fn, out)`	Same for floats	outputTensor	FLOAT32
`int32ElementWithTensorArithmeticInplace(t, x, fn)`	op(t[i], x) → written back into t	tensor	INT32
`floatElementWithTensorArithmeticInplace(t, x, fn)`	Same for floats	tensor	FLOAT32

Function pointer types:

Type name	Signature	Used by
`int32ElementArithmeticFunc_t`	int32_t fn(int32_t a, int32_t b)	all int32 arithmetic functions
`floatElementArithmeticFunc_t`	float fn(float a, float b)	all float arithmetic functions